# Sorting Benchmark Lab: Insertion, Merge, Quick and Python sorted()

CS314 · Divide and Conquer · Mini project report

## 1. Problem

Which sorting algorithm should we use, and how does the input order change the answer? We compare **insertion sort**, **merge sort**, **quicksort** (last-element pivot, as in the lecture), **quicksort with a random pivot** (an extra) and Python's built-in **sorted()** (Timsort) on **sorted**, **reversed** and **random** data, and check whether measured running times match theory.

## 2. Method

All algorithms are in `sorts.py` and return a new sorted list. They pass unit tests on empty, single, duplicate, negative, sorted, reversed and random inputs, on the lecture examples (slide 94), and on stability of merge sort. Timing uses `time.perf_counter`, the median of 5, 3 or 2 repeats depending on n, with a fixed random seed. Sizes are n = 1,000 to 100,000. Because pure Python is slow, algorithms that are quadratic on a given input (insertion on reversed/random, last-pivot quicksort on sorted/reversed) were run only up to n = 10,000. Quicksort uses an explicit stack, so its worst case is slow but cannot overflow the recursion limit.

## 3. Complexity analysis

**Insertion sort**: best case $\Theta(n)$ (sorted: one comparison, no shifts per element); worst case $\Theta(n^2)$ with n(n-1)/2 shifts (reversed); average $\Theta(n^2)$ with about half the shifts of the worst case (each element moves about halfway). **Merge sort**: T(n) = 2T(n/2) + n, so $\Theta(n \log n)$ for every input (Master Theorem, case 2), using O(n) extra space. **Quicksort (last pivot)**: average $\Theta(n \log n)$, but on sorted or reversed data the pivot is always the minimum or maximum, so T(n) = T(n-1) + n = $\Theta(n^2)$. A random pivot makes this bad split very unlikely. **Timsort** finds already-ordered runs, so it is O(n) on sorted or reversed input and O(n log n) otherwise.

## 4. Experiments and results

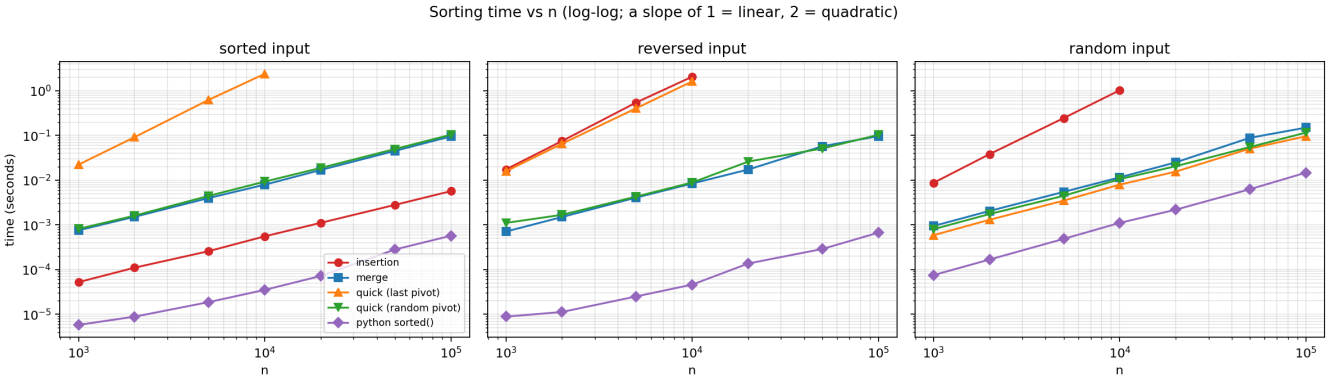



Figure 1. Median time versus n on log-log axes. Parallel lines with slope 1 are linear / n log n; slope 2 is quadratic.

| Algorithm | Sorted | Reversed | Random |
|---|---|---|---|
| insertion | 1.01 | 2.06 | 2.04 |
| merge | 1.06 | 1.09 | 1.13 |
| quick (last pivot) | 2.03 | 1.99 | 1.11 |
| quick (random pivot) | 1.06 | 1.07 | 1.07 |
| python sorted() | 1.09 | 1.06 | 1.13 |

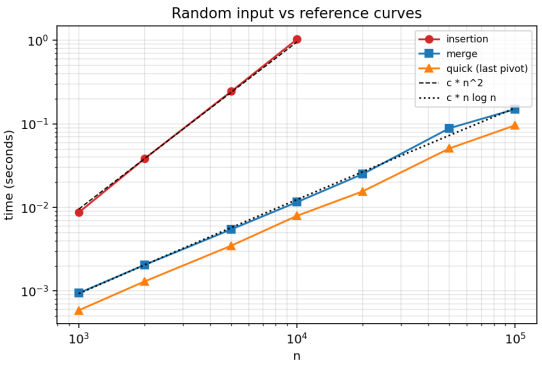



Table 1 (left). Estimated growth exponent: slope of log(time) vs log(n) for n ≥ 2,000; shaded cells are the quadratic cases. Figure 2 (right). Random input with $c \cdot n^2$ and $c \cdot n \log n$ reference curves.

## Main observations

- **Best case.** Insertion sort on sorted data has slope 1.01: doubling n from 50,000 to 100,000 multiplied the time by 2.03. It was the fastest of our own implementations there (0.006 s at n = 100,000, versus 0.096 s for merge sort).
- **Worst and average case.** Insertion sort has slope about 2 on both reversed and random data. At n = 10,000 the random case took 1.03 s and the reversed case 2.07 s: a ratio of 0.50, matching the theory that random input needs about half the shifts of the worst case. At n = 10,000 insertion sort on random data was 89 times slower than merge sort.

- **Last-pivot quicksort is not safe.** On random data it was the fastest of our own sorts (0.096 s at n = 100,000), but on sorted data it had slope 2.03 and took 2.40 s at n = 10,000, slower than insertion sort on reversed data. With a random pivot the same inputs needed only 0.10 s at n = 100,000.
- **Merge sort is predictable.** Slopes of 1.06 to 1.13 on all three inputs, consistent with n log n (which shows a slope slightly above 1 in this range), and almost the same time on every input order.
- **Built-in sorted()** was about 10 times faster than our merge sort on random data (0.015 s versus 0.150 s) and took under 1 ms on sorted and reversed data, because it is implemented in C and exploits existing runs.

## 5. Conclusion and limitations

The measurements agree with the theory: $\Theta(n)$ for insertion sort on sorted data, $\Theta(n^2)$ on reversed and random data, $\Theta(n \log n)$ for merge sort on every input, and a quadratic worst case for last-pivot quicksort that random pivoting removes. In practice, use the built-in `sorted()`; if writing our own, prefer merge sort or randomised quicksort, and use insertion sort only for tiny or nearly sorted arrays. **Limitations:** our smallest size is n = 1,000, so we did not measure the small-n region ($n \leq 20$-$50$) where insertion sort is usually competitive, and we did not test nearly-sorted inputs; quadratic cases stop at n = 10,000; all numbers come from one machine and pure Python, so absolute times are not portable, only the growth trends.